

A Lean 4 Certificate of Furber et al (CSL 2026) Interpreting Lambda Calculus in Domain-Valued Random Variables

Lars Warren Ericson¹

¹Catskills Research Company

¹<https://github.com/catskillsresearch/scott2026>

¹lars.ericson@catskillsresearch.com

September 14, 2026

ORCID: 0000-0001-8299-9361

Primary Category: cs.LO (Logic in Computer Science)

Secondary Categories: math.LO (Logic); cs.PL (Programming Languages)

Abstract

This note records a Lean 4 / Mathlib formalization of Furber, Mardare, Panangaden, and Scott’s CSL 2026 paper *Interpreting Lambda Calculus in Domain-Valued Random Variables*. The paper develops Boolean-valued set theory and domain theory and interprets the λ -calculus via domain-valued random variables. The sorry-free `Scott2026/` library (~38,000 lines) covers V^A , the internal Engeler model, Theorems 26 and 30, Corollary 34, Lemma 35 / Proposition 36, the measure algebra $A(X) = \Sigma/\mathcal{N}$, Propositions 39–42, Theorem 43, and Proposition 44. Continuous-lattice background is vendored from `scott1972` (`vendor/scott1972`). Deliberate `sorry`s appear only in the Mathlib-only `Challenge.lean` (`cs12026_internal_interpretation`, `cs12026`, `proposition_36_i`); `Solution.lean` re-exports kernel-checked proofs. There are no project axioms beyond Mathlib’s classical footprint. The paper’s authors were not involved in or endorsing this work. Lean was written by AI agents under the author’s direction and review. Proof summaries include short Lean fragments; the full library is indexed with links to <https://github.com/catskillsresearch/scott2026>.

1 Introduction

Boolean-valued models of set theory were introduced so that independence results—most famously the continuum hypothesis—could be read as statements about names and Boolean truth values rather than as metatheoretic constructions of forcing extensions [Sco67, Bell, Jech]. Scott later asked whether the same idea could serve *semantics*: if the Boolean algebra A is a measure algebra, then an A -valued set is a random variable taking values in a domain, and the internal language of V^A becomes a language of almost-sure statements about higher-type computation.

The CSL 2026 paper carries that program through for the untyped λ -calculus. It rebuilds, internally, the ingredients of a reflexive domain: A -posets, Scott continuity, the Engeler graph model $\mathcal{P}^A(\tilde{E})$, Church numerals, and a many-one comparison of subsets of $\mathbb{N}$ defined by λ -terms. The application (Theorem 43) produces two subsets of $\mathbb{N}$ that are incomparable under that λ -definable preorder, using independent fair coins as the underlying negligibility space. Proposition 44 records that one cannot shortcut the argument by treating $L^0(X; \mathcal{P}(Y))$ as a continuous dcpo: when $A(X)$ is non-atomic, it is not.

This formalization follows the paper’s numbered statements in dependency order. Weaker or special-case Lean theorems keep their original names; paper-type statements are introduced beside them (`theorem_30_va` beside `theorem_30`, `lemma_35_of` beside Engeler `lemma_35`, and so on). The working source is `sources/Scott2026.pdf`, with a searchable transcription in `sources/Scott2026_vision.md`. A related long version is [arXiv:2112.06339](https://arxiv.org/abs/2112.06339).

Three methodological choices are worth stating up front.

1. **Extensional ZFC boundary.** Ground sets are read through Mathlib’s `ZFSet` quotient. Raw `PSet` representatives remain an implementation device, used only where structural recursion needs them.
2. **Nontrivial Boolean algebras.** The paper’s distinctions ($\perp \neq \top$, numeral separation, strictness of $\text{Oid}(D)$) have no intended meaning in the one-element algebra. Lean states `[Nontrivial A]` exactly where those distinctions appear.
3. **Measure algebra as $\Sigma/\mathcal{N}$.** The paper’s $A(X)$ is the quotient of *measurable* sets by null sets (`MeasureAlgebra`). An all-sets presentation (`NegligibilitySpace.measurableRep`) is retained but is strictly stronger than Borel coin space can inhabit.

The formalization is **classical**. Every completed proof reports the standard mathlib footprint `[propext, Classical.choice, Quot.sound]`.

2 Scope and layout

The sorry-free development is about 38,000 lines across 38 Lean modules under `Scott2026/`, plus the vendored Scott 1972 continuous-lattice library. Deliberate proof holes occur only in the Mathlib-only `Challenge.lean`. That challenge face is a Mathlib-expressible subset of Proposition 27; the remaining paper-type theorems are kernel-checked in `Scott2026/` and re-exported by `Solution.lean`.

Module cluster	Role
VA, ExtensionalVA, Oid, Setoid, RelFun InternalDomain, InternalInterp, Internal Eval*	Boolean-valued names, A -setoids, mixing Internal Definition 19 and the Theorem 26 evaluator
Theorem26, Theorem30Internal, Corollary34	Internal interpretation and Engeler in V^A
Domain, Lambda, Interp, Lemma35General, Prop36	Ground reflexive deposes, Church calculus, oracles
Random, Coin	Measure algebra, L^0 , G_X , Propositions 42–44
Basic	Re-exports and inventory comments
Challenge / Solution	Mathlib-only challenge statements vs sorry-free proofs

`Scott1972` is compiled from `vendor/scott1972 (srcDir; pin a198b6e, see vendor/FROZEN.txt)`, not as a Lake path/git dependency.

3 How the proofs use Mathlib

The paper rebuilds a large fragment of Boolean-valued set theory and domain theory. Mathlib supplies the algebraic and measure-theoretic substrate; the paper-specific language (A -names, internal Scott continuity, Church combinators, G_X) is developed in `Scott2026/`.

Order and Boolean algebra. Complete Boolean algebras, Heyting implication, and complete lattices come from `Mathlib.Order.CompleteBooleanAlgebra` and `Mathlib.Order.CompleteLattice`. Internal predicates (`isFunctionB`, `isScottContinuousB`, `isReflexiveDcpoB`) are A -valued terms in that algebra, not Prop-valued class-instance searches.

ZFC presentations. `Mathlib.PSet / ZFSet` (`Mathlib.SetTheory.ZFC`) give a well-founded model of sets. We use `PSet` only for recursion (Church numerals as `PSet.omega`, pairing codes) and expose `ZFSet / checkExt` at the paper boundary, with representation-independence lemmas so two equivalent pre-sets cannot create a spurious failure of strictness.

Domain theory. Scott-open sets, way-below, and the injective extension operator `scottExtend` (Scott 1972, Props 2.11 / 3.8) come from the vendored `Scott1972` library, which itself sits on `Mathlib` topology (`IsEmbedding`, `Continuous`, Scott topology). Lemma 35(ii) applies `scottExtend` to the discrete numeral subspace and feeds the extension to `ReflexiveDcpo.lam`.

Measure theory. Fair coins are `Mathlib` Bernoulli measures on `Bool` (extended by `Measure.infinitePi` to 2^ω and by `Measure.prod` to $2^\omega \times 2^\omega$). Nullity, countable unions, and `toMeasurable` are the usual `MeasureTheory` lemmas. The paper algebra `MeasureAlgebra`, parameterized by a measure μ , is a `Quotient` of $\{s \text{ // MeasurableSet } s\}$ by $\mu(s \triangle t) = 0$, with a `CompleteBooleanAlgebra` instance under `[IsFiniteMeasure  $\mu$ ]` (countable-chain-condition essential suprema).

Combinatorics of syntax. Capture-avoiding substitution needs `[Infinite Var]`. Countability of closed λ -terms for Theorem 43 uses an injective `Nat.pair` encoding (`Mathlib.Data.Nat.Pairing`).

What `Mathlib` does *not* provide, and what this library therefore builds, is the internal language of V^A : Boolean-valued membership and equality on names, the Oid equivalence, the relational evaluator of Definition 25, and the $A(X)$ -poset L^0 .

4 Proof dependency structure

The paper’s numbered results are not independent modules. The Lean import graph follows the published order: extensional names, then the internal evaluator, then Engeler in V^A , then numerals and oracles, then the measure-algebra example.

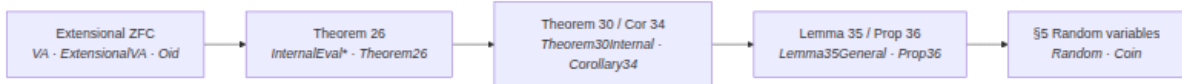


Figure 1: High-level dependency order of the formalized paper results.

5 Theorem inventory

Paper names and Lean names. Weaker or special-case theorems keep their original names beside the paper-type theorems.

Paper	Paper-type Lean name	Weaker / special-case name
Proposition 27	<code>proposition_27 / proposition_27_finite_* wrappers</code>	<code>finite_subsets_countable</code>
Theorem 26	<code>theorem26Full / theorem26Pure / theorem26Pure_sound</code>	<code>theorem_26</code> (Engeler carrier)
Theorem 30	<code>theorem_30_va / theorem_30_internalModel</code>	<code>theorem_30</code> (canonical-carrier)

Paper	Paper-type Lean name	Weaker / special-case name
Corollary 34	corollary_34	corollary_34_check
Lemma 35	lemma_35_of / lemma_35_i_of / lemma_35_ii_of	lemma_35 (Engeler)
Proposition 36	proposition_36_of	proposition_36 (Engeler)
Proposition 42	proposition_42_algebra	proposition_42 (raw coin Measure)
Theorem 43	theorem_43_paper	theorem_43 (external oracles)
Proposition 44	proposition_44_coin / _measure / _dcpo	proposition_44 (all-sets algebra)

Further named results: `definition_25 / interp`, `lemma_31`, `definition_32`, `proposition_33`, `churchNotN / churchTest / churchWithNumerals`, `MeasureAlgebra / coinAlgebra / coinS1 / coinS2`, `G_X_measure`, `proposition_39_measure`, `proposition_40_measure`, `lemma_41_measure / lemma_41_algebra`, `not_isAtomic_coinAlgebra`.

Compared declarations in `Challenge.lean`: `csl2026_internal_interpretation` (the Boolean-valued interpretation and internal Engeler consequences of Theorem 26 and Corollary 34), `csl2026` (Theorem 43), and `proposition_36_i`. The Solution instantiates the internal-interpretation face with `interpClosedVA`; the proof of `csl2026` uses `csl2026_capstones` (`theorem26Full`, `corollary_34`, `theorem_43_paper`). Representative challenge statements:

Lean 4 source

```

theorem csl2026_internal_interpretation :
  internal_interpretation_statement := by
  sorry

theorem csl2026 : exists T_1 T_2 : Set Nat,
  not proposition_36_i T_1 T_2 /\ not proposition_36_i T_2 T_1 := by
  sorry

def proposition_36_i (S_1 S_2 : Set Nat) : Prop :=
  exists M : Lam Nat, exists hM : MapsNumerals M,
  forall n, n in S_1 <-> mapsNumeralsFun M hM n in S_2

```

6 Proof notes

This section records strategy where the mechanization has to say more than the published text, and shows a short Lean fragment when the library proof is short. Longer constructions are cited by file.

6.1 Boolean-valued set theory and Oid

Jech’s lemmas on mixing, fullness, and Δ_0 reflection are `jech_lemma_14_18`, `theorem_1_iii`, and `jech_lemma_14_21`. The categorical equivalence between A -setoids and extensional names is `setAEquivSetoidR`. Completeness, totality, and strictness of $\text{Oid}(\mathcal{P}^A(X))$ are `theorem_17_va_*` / `proposition_28_va_*`. These are the substrate for every later internal statement: an internal function name is an A -valued relation that is single-valued and total at Boolean degree $\top$, then packaged as a `SetoidFHom` by Definition 16.

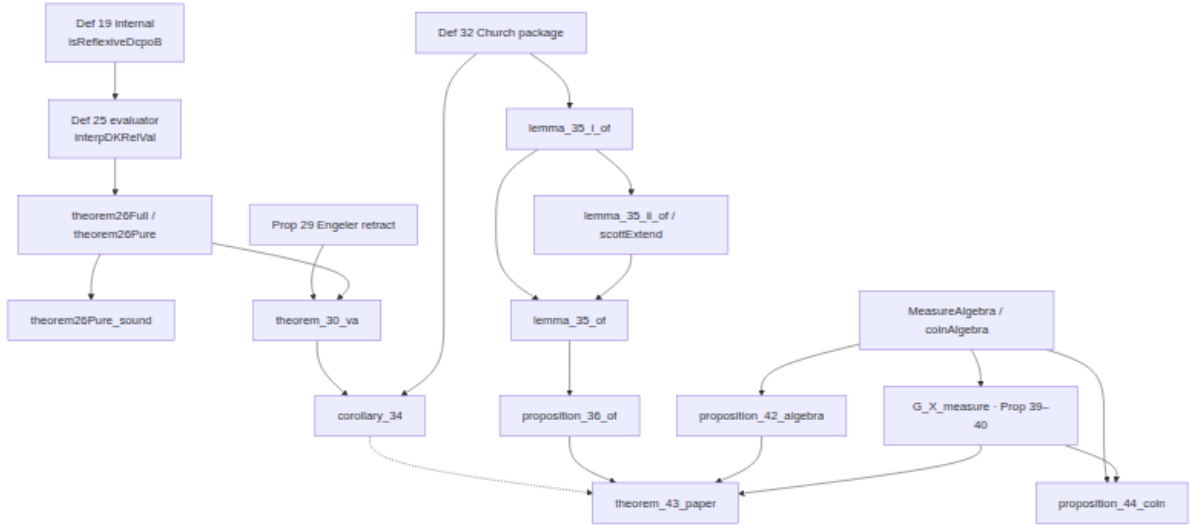


Figure 2: Internal interpretation, Engeler model, numerals, and the random-variable capstone (Theorems 26–43).

6.2 Theorem 26 — internal interpretation

Definition 25 interprets $\Lambda(D, \text{Var}, \mathfrak{R})$ by recursion on terms, in the internal language. The difficulty the paper leaves implicit is that abstraction must survive *fuzzy* environment updates: the body is interpreted along a pointwise Scott-continuous family, not a crisp substitution. The library therefore proves a fundamental lemma quantifying over `IsPointwiseFamily` environments (`InternalEvalFamily`, `InternalEvalComplete`), identifies application and abstraction body graphs as Scott-continuous maps into the internal function space C , and only then packs an evaluator name `interpDKGraph` (`InternalEvalPack`).

`theorem26Full` is then Definition 16 applied to that name:

Lean 4 source

```
noncomputable def theorem26Full
  (? : InternalReflexiveModel (A := A))
  (V K : AName.{u} A) (hK : subsetB K ?.D = top)
  (hV : (oid V).IsTotal)
  (? : RelFun (oid V) (oid ?.D)) :
  SetoidFHom (oid (lamDKB ?.D V K)) (oid ?.D) :=
theorem26Full0fFunctionName ?.D V K
  (InternalReflexiveModel.interpDKGraph ? V K hK hV ?)
  (InternalReflexiveModel.interpDKGraph_isFunctionB ? V K hK hV ?)
  ?.complete
```

Soundness `theorem26Pure_sound` is the full `LamEq` theory (reflexivity, symmetry, transitivity, congruence, ξ , capture-avoiding β , α). The extra hypothesis actually needed is `OldSeparated V` (distinct variable keys are Boolean-unequal), together with `[Infinite V.idx]` for `Lam.substCA`. The Engeler-only `theorem_26` on `Set ℕ` is kept as a weaker packaging. Full proof: `Scott2026/InternalEvalComplete.lean`, `InternalEvalPack.lean`, `Theorem26.lean`.

6.3 Theorem 30 and Corollary 34 — Engeler in V^A

Proposition 29’s `app/lam` pair is internalized as names `engelerFun` and `engelerLamB` on $D = \mathcal{P}^A(\tilde{\omega})$. The retract `fun ∘ lam = idC` uses Scott continuity of $G \in C$ on the directed family of finite subsets, plus membership of ground finite sets in the checked ω . The six-tuple of Definition 19 then has Boolean value $\top$:

The packaging of Corollary 34 is a conjunction, so the library proof is a tuple of already-proved conjuncts.

Lean 4 source

```
theorem corollary_34 [Nontrivial A] :
  isReflexiveDcpoB engelerD engelerR engelerC engelerQ
    engelerFun engelerLamB = top /\
  isContinuousLatticeSubsetB engelerD = top /\
  isBaseSubsetB (pfinB (checkExt PSet.omega)) engelerD = top /\
  Corollary34Check /\
  (eqB (child? (interpClosedVA churchTrue))
    (child? (interpClosedVA churchFalse)) = False) /\
  (forall n m, eqB (child? (interpClosedVA (churchNum n)))
    (child? (interpClosedVA (churchNum m)))) = top -> n = m) :=
<theorem_30_va, isContinuousLatticeSubsetB_powerB _,
  isBaseSubsetB_pfinB_powerB _, corollary_34_check,
  eqB_interpClosedVA_churchTrue_churchFalse,
  fun n m h => churchNum_interpClosedVA_injective h>
```

([Nontrivial A] is required for checkExt and for the last two conjuncts.) Full proof: Scott2026/Theorem30Internal.lean, Corollary34.lean.

6.4 Lemma 35 — discrete numerals and oracles

The paper derives closed tests $\mathbf{m}^?$ from Definition 32's *if*, *pred*, and *0?*, then uses Scott-open separators and (for part (ii)) injectivity of continuous lattices. In Lean, `churchNotN` and `churchTest` live on $\mathbb{N}$ so that `interpClosed_sound_full` applies (`[Infinite Var]`). Transfer of the λ -equations is one rewrite:

Lean 4 source

```
theorem interpClosed_churchNotN_true {D : Type u} [CompleteLattice D]
  (R : ReflexiveDcpo D) :
  R.app (interpClosed R churchNotN) (interpClosed R churchTrueN) =
    interpClosed R churchFalseN := by
  rw [ <- interpClosed_app]
  exact interpClosed_sound_full (Var := Nat) R churchNotN_true
```

N numeral injectivity is the paper's function argument: if $\llbracket c_m \rrbracket = \llbracket c_n \rrbracket$ then $\mathbf{fun}(\llbracket m^? \rrbracket)$ cannot send one to $\top$ and the other to $\perp$ (`numeral_inj_of_churchTest`). Part (ii) embeds the numeral subspace, notes that discreteness makes $g(\llbracket c_n \rrbracket) = \chi_A^D(n)$ continuous, and extends by `scottExtend` (Scott 1972). The retract identity `lemma_35_ii_of_extension` turns $\mathbf{lam} \bar{g}$ into the oracle. Full proof: Scott2026/Lambda.lean, Lemma35General.lean.

6.5 Proposition 36 — λ -definable many-one comparison

On a reflexive *continuous* lattice with Church numerals, paper (i) (some oracles) and (ii) (all oracles) are equivalent because Lemma 35(ii) supplies at least one pair. The library proof is exactly that pair of implications; `ManyOneLe` (an arbitrary $f : \mathbb{N} \rightarrow \mathbb{N}$) is kept separate, and the converse `ManyOneLe` $\rightarrow$ `proposition_36_i` is not claimed (`exists_nat_fun_not_lambda_definable`).

Lean 4 source

```
theorem proposition_36_of {D : Type*} [CompleteLattice D]
  (R : ReflexiveDcpo D)
  (hbool : interpClosed R churchTrueN != interpClosed R churchFalseN)
  (hcont : Scott1972.ContinuousLattice.IsContinuousLattice D)
```

```

(S_1 S_2 : Set Nat) :
proposition_36_i_of R hbool S_1 S_2 <->
  proposition_36_ii_of R hbool S_1 S_2 :=
<proposition_36_ii_of_i_of R hbool,
  proposition_36_i_of_ii_of R hbool hcont>

```

6.6 Measure algebra, G_X , and Proposition 42

MeasureAlgebra quotients measurable sets. The Boolean value of a measurable event is $\perp$ exactly when the event is null:

Lean 4 source

```

theorem mk_eq_bot [IsFiniteMeasure ?] {s : Set X}
  {hs : MeasurableSet s} :
  mk ? s hs = (False : MeasureAlgebra ?) <-> ? s = 0 := by
  rw [ <- mk_bot, mk_eq_iff, ae_empty_iff ]

```

Equation (5) is $\text{coinS1 } n = \text{mk } (D1 \ n \ \text{true})$ (and likewise coinS2). Proposition 39–40 and Lemma 41 are restated on this algebra as `proposition_39_measure`, `proposition_40_measure`, `lemma_41_measure`; the un-suffixed names remain the all-sets `AssociatedAlgebra` forms. Proposition 42’s algebra form is four Boolean values equal to $\perp$: no finite standard preimage of S_1 or S_2 , and neither $S_1 = \check{f}^{-1}(S_2)$ nor the swap. Each conjunct is a Mathlib null-set calculation already in `proposition_42`, lifted by `mk_eq_bot`.

Lean 4 source

```

theorem proposition_42_algebra (f : Nat -> Nat) :
  (( bigcup K : Finset Nat, ? n, (coinS1 n ? checkSet K (f n)) sqcap
    (checkSet K (f n) ? coinS1 n) ) = False) /\
  (( bigcup K : Finset Nat, ? n, (coinS2 n ? checkSet K (f n)) sqcap
    (checkSet K (f n) ? coinS2 n) ) = False) /\
  ((? n, (coinS1 n ? coinS2 (f n)) sqcap (coinS2 (f n) ? coinS1 n) ) = False) /\
  ((? n, (coinS2 n ? coinS1 (f n)) sqcap (coinS1 (f n) ? coinS2 n) ) = False) :=
<iSup_eq_bot.mpr fun K => coinS1_finite_preimage_eq_bot f K,
  iSup_eq_bot.mpr fun K => coinS2_finite_preimage_eq_bot f K,
  coinS1_preimage_S2_eq_bot f, coinS2_preimage_S1_eq_bot f>

```

The finite- K join is over `Finset N`, not the full internal $\mathcal{P}_{\text{fin}}^A(\tilde{\omega})$. Full proof: `Scott2026/Random.lean, Coin.lean`.

6.7 Theorem 43 — incomparable λ -degrees

The paper applies Theorem 1 to Lemma 35(ii) *inside* V^A , transports along G_X^{-1} , and intersects countably many conull sets. The library proof `theorem_43_paper` follows that *mathematical* recipe with one honest substitution of tools: oracles are fiberwise ground Lemma 35(ii) (`chi Oracle` on the bit-sets), mixed as L^0 maps and pushed through `G_X_measure`, rather than a direct `theorem_1_iii` witness on an internalized Lemma 35 formula. Application is the existing Engeler operation `engelerAppA`. After Proposition 42 makes each numeral-mapping M null, countability of `Lam N (lamEncode_injective)` and `measure_univ = 1` produce a point x ; $T_i = \{n \mid a_i(x) \cdot \llbracket c_n \rrbracket = \llbracket T \rrbracket\}$ and Proposition 36 give both non-reducibility conclusions.

The statement is short; the construction of the conull set is not, so this note does not copy the body. The capstone type matches the library proof:

Lean 4 source

```

theorem theorem_43_paper :
  exists T_1 T_2 : Set Nat, not proposition_36_i T_1 T_2 /\ not proposition_36_i T_2 T_1

```

After `exists_mem_paperGoodSet` the library defines T_i as the set of n with $a_i(x) \cdot \llbracket c_n \rrbracket = \llbracket \top \rrbracket$ and finishes with `proposition_36_i` in `Scott2026/Coin.lean`. The external-oracle theorem `theorem_43` is unchanged and has the same type.

6.8 Proposition 44 — L^0 is not a continuous dcpo

On a complete lattice the paper’s “continuous dcpo” is the existing way-below-sup predicate (`IsContinuousDcpo` aliases `IsContinuousLattice`). Transport along G_X reduces the claim to: the function space of a non-atomic complete Boolean algebra is not a continuous lattice. Fair-coin `coinAlgebra` is non-atomic because any positive class splits along a first-sequence bit (or else determines that sequence and is null). Full proof: `proposition_44_coin` in `Coin.lean`; the `AssociatedAlgebra` form `proposition_44` is kept.

7 Source fidelity

Working source: `sources/Scott2026.pdf`. Vision transcription: `sources/Scott2026_vision.md` (produced by `scripts/ocr_pdf_pipeline.sh`).

7.1 Foundational convention

We read the paper’s ground sets extensionally, in ZFC. The Lean formalization therefore exposes ground-set parameters through `Mathlib’s ZFSet`; `PSet` is used only as an implementation-level well-founded presentation behind proved representation-independence lemmas. Boolean names use the quotient domain `AName.Dom`, and checked sets use `checkExt`, so duplicate pre-set presentations cannot create spurious failures of strictness.

We also believe the paper implicitly assumes that the complete Boolean algebra A is non-trivial. Statements that distinguish Boolean truth values or require $\perp \neq \top$ therefore state `[Nontrivial A]` explicitly. This is necessary: for the one-element Boolean algebra all Boolean equalities simultaneously have values $\top$ and $\perp$, so the paper’s numeral-separation and strictness conclusions cannot have their intended meaning.

7.2 Recorded divergences

- `proposition_42_algebra` `finite-K` is a `Finset` join (the paper’s fuzzy `pfinB` exists-quantifier is not fully internalized).
- `theorem_43_paper` uses fiberwise Lemma 35(ii) mixed in L^0/G_X `measure`, not a direct `theorem_1_iii` hookup; `theorem_43` remains the external-oracle form.
- All-sets `NegligibilitySpace.measurableRep / ofMeasure hN` is stronger than paper $\Sigma/\mathcal{N}$; paper $A(X)$ is `MeasureAlgebra`.
- Engeler-only `lemma_35 / proposition_36 / proposition_42 / theorem_43` keep weaker or special-case names.
- `Challenge.lean` compares `csl2026_internal_interpretation`, `csl2026` (Theorem 43), and `proposition_36_i`. The project-facing capstones `theorem26Full`, `corollary_34`, and `theorem_43_paper` remain kernel-checked in `Scott2026/`.

8 Build

The repository pins Lean / `mathlib v4.33.0` (`lean-toolchain`).

```
lake exe cache get
lake build
bash scripts/build_arxiv_pdf.sh
```

Regenerate `arxiv_with_code.md` when `arxiv.md` or listed sources change: `bash scripts/generate_arxiv_with_code.sh`.

9 Acknowledgments

Thanks to **Prof. Dana Scott** (Carnegie Mellon University) for bringing the CSL 2026 paper to the author’s attention. Scott did not participate in this work and does not endorse it.

9.1 AI-assisted development

Lean in this repository was written by AI agents under the author’s direction and review. The human author retains sole responsibility for the mathematical content, the formalization route, and every formal claim. **No large language model is listed as a co-author.**

We gratefully acknowledge assistance from the following tools:

- **GPT-5.6 [Gpt26]** — OpenAI’s GPT-5.6 family in Cursor (Sol variant) for Lean 4 / mathlib formalization, `lake build` repair, and long-running agent loops on `Scott2026/`. Generated Lean was provisional until it compiled under the pinned toolchain.
- **Anthropic Claude Fable 5 [Fab26]** — selective use in Cursor for the heaviest proof work and long-horizon formalization. Every emitted proof term was checked by the Lean kernel.
- **Anthropic Claude Opus 5 [Opu26]** — day-to-day formalization and proof-engineering in Cursor: inventory and narrative maintenance, module wiring, and medium-to-hard Lean obligations where the proof strategy was already fixed.
- **Cursor Grok 4.6 [Grk26]** — primary SpaceXAI / Cursor agent for Lean 4 / mathlib formalization, `lake build` repair, vision-OCR transcription, drafting this narrative (`arxiv.md`), and tracking the formalized inventory.
- **Cursor Composer 2.5 [Cmp25]** — routine multi-step work: module scaffolding, dependency-ordered wiring of `Scott2026/`, documentation, and medium proof obligations where the strategy was already fixed.

List of Figures

1	High-level dependency order of the formalized paper results.	3
2	Internal interpretation, Engeler model, numerals, and the random-variable capstone (Theorems 26–43).	5

10 References

- [Gpt26] OpenAI. *GPT-5.6* (Sol variant as integrated in Cursor). Model documentation, <https://cursor.com/docs/models/gpt-5-6-sol> (accessed 2026).
- [Fab26] Anthropic. *Claude Fable 5*. Announcement, <https://www.anthropic.com/news/claude-fable-5-mythos-5>; system card, <https://www-cdn.anthropic.com/2f9323abbcc4abe219577539efe19a623c9ca2bd/Claude%20Fable%205%20%20Claude%20Mythos%205%20System%20Card.pdf>; Cursor model page, <https://cursor.com/docs/models/claude-fable-5> (accessed 2026).
- [Opu26] Anthropic. *Claude Opus 5*. Announcement, <https://www.anthropic.com/research/claude-opus-5>; model documentation as integrated in Cursor, <https://cursor.com/docs/models/claude-opus-5> (accessed 2026).
- [Grk26] SpaceXAI and Anysphere, Inc. *Grok 4.6*. Official model card, <https://media.x.ai/v1/website/card-7f81d41b.pdf>; developer documentation, <https://docs.x.ai/developers/models/grok-4.6>; Cursor model page, <https://cursor.com/docs/models/grok-4-6>; Cursor announcement, <https://cursor.com/blog/grok-4-6> (accessed 2026).
- [Cmp25] Anysphere, Inc. *Composer 2.5*. Model announcement and documentation, <https://cursor.com/blog/composer-2-5>; model card as integrated in Cursor, <https://cursor.com/docs/models/composer-2-5> (accessed 2026).

11 License and source PDF

Original Lean and author-written docs: Apache-2.0. `sources/Scott2026.pdf` and `sources/JechSetTheory2003.pdf` are **not** Apache-2.0; see NOTICE and `sources/README.md`.

A Lean module index

Checked by `lake build`. Complete sources: <https://github.com/catskillsresearch/scott2026>. Each row links to the corresponding file on GitHub.

Role	File
Root import graph	Scott2026.lean
Boolean connectives and Hilbert calculus	Scott2026/BooleanLogic.lean
A-setoids and mixing	Scott2026/Setoid.lean
Relational functionals	Scott2026/RelFun.lean
Setoid categories	Scott2026/Categories.lean
A-subsets and checks	Scott2026/PowerSet.lean
Oid equivalence	Scott2026/Oid.lean
<code>Set_A</code>	Scott2026/SetCategory.lean
Oid strictness and completeness	Scott2026/OidEssential.lean
Raw-power regression tests	Scott2026/RawPowerStrict.lean
Boolean-valued names (Jech / Theorem 1)	Scott2026/VA.lean
Extensional ZFC boundary	Scott2026/ExtensionalVA.lean
Internal way-below / continuous lattices	Scott2026/InternalDomain.lean
Propositions 27–28	Scott2026/Proposition28.lean

Role	File
Ground reflexive dpos	Scott2026/Domain.lean
λ -syntax and Definition 32 combinators	Scott2026/Lambda.lean
Ground interpretation (Definition 25)	Scott2026/Interp.lean
Engeler model (Proposition 29)	Scott2026/Engeler.lean
Internal Engeler application	Scott2026/EngelerVA.lean
Internal reflexive dpos	Scott2026/ReflexiveVA.lean
Internal pure λ -syntax	Scott2026/LambdaVA.lean
Constant-bearing syntax	Scott2026/LambdaConstVA.lean
Ground constant interpretation	Scott2026/InterpConst.lean
Internal interpretation (Engeler carrier)	Scott2026/InterpVA.lean
Internal constant interpretation	Scott2026/InterpConstVA.lean
Relational calculus for evaluation	Scott2026/InternalInterp.lean
Internal evaluator recursion	Scott2026/InternalEval.lean
Pointwise environment families	Scott2026/InternalEvalFamily.lean
Scott continuity of app/abs	Scott2026/InternalEvalComplete.lean
Evaluator name and Theorem 26 packing	Scott2026/InternalEvalPack.lean
Exact Theorem 26	Scott2026/Theorem26.lean
Exact Theorem 30	Scott2026/Theorem30Internal.lean
Lemma 31 (check/VA agreement)	Scott2026/Lemma31.lean
Corollary 34	Scott2026/Corollary34.lean
General Lemma 35	Scott2026/Lemma35General.lean
Proposition 36	Scott2026/Prop36.lean
Measure algebra, L^0 , G_X	Scott2026/Random.lean
Coin space; Props 42–44; Theorem 43	Scott2026/Coin.lean
Re-exports and inventory	Scott2026/Basic.lean

Primary source (PDF): [sources/Scott2026.pdf](#) — Furber, Mardare, Panangaden, and Scott, *Interpreting Lambda Calculus in Domain-Valued Random Variables* (LIPIcs, CSL 2026).

Total: 39 modules, 38108 lines of Lean in [Scott2026/](#).